Comunicação Cliente-Servidor via UDP no Godot

Visão geral da implementação

Nesta etapa, a comunicação cliente-servidor foi organizada em torno de mensagens estruturadas, enviadas por UDP.
O cliente envia ao servidor:

- Seu registro inicial (nome de jogador).
- Sua posição atual (a cada frame).

O servidor mantém uma tabela de jogadores conectados e suas informações (nome, IP, porta e posição).
Em cada iteração, o servidor responde ao cliente com um snapshot global do estado de todos os jogadores, permitindo que cada cliente reconstrua o ambiente remoto em tempo real.

Essa etapa refina a versão anterior (troca crua de dados via sockets) adicionando:

- Mensagens semânticas padronizadas (REGISTER, POS, PLAYER_POSITIONS).
- Armazenamento estruturado no servidor (`players` como dicionário).
- Gerenciamento dinâmico de instâncias de jogadores remotos no cliente.

Projeto de arquitetura de mensagem/fluxo

Tipos de mensagens definidas

O sistema implementa três tipos principais:

1. **Comandos de cliente → servidor**
    a. `REGISTER:<nome>` → identifica o jogador recém-conectado.
    b. `POS:<x>:<y>` → atualiza a posição do jogador.
2. **Eventos de servidor → cliente**
    a. `PLAYER_POSITIONS:<lista>` → snapshot do estado global dos jogadores (nome + posição de todos os conectados).
3. **Mensagens não reconhecidas**

a. Tratadas pelo cliente/servidor como “mensagem desconhecida”, apenas para debug.

Estrutura / formato das mensagens-chave

- Registro

```
REGISTER:jogador_local1
```

  - Campo 1: comando
  - Campo 2: nome do jogador

- Atualização de posição

```
POS:123.5:456.2
```

  - Campo 1: comando
  - Campo 2: X
  - Campo 3: Y
- Estado global

```
PLAYER_POSITIONS:id1:JogadorA:100.0:200.0;id2:JogadorB:300.0:400.0
```

  - Campo 1: comando
  - Campo 2: lista de jogadores separados por `;`
  - Para cada jogador: `id:nome:x:y`

Fluxos de comunicação principais

- Fluxo de registro
  - Cliente envia `REGISTER:<nome>` → servidor cria/atualiza entrada no dicionário `players`.
  - Impacto: jogador passa a existir no mundo compartilhado.
- Fluxo de atualização de posição
  - Cliente envia `POS:<x>:<y>` a cada frame.
  - Servidor atualiza `players[id]["position"]`.
- Fluxo de sincronização global

- Após processar qualquer mensagem recebida, servidor responde com `PLAYER_POSITIONS:<lista>`.
- Cliente atualiza ou instancia os jogadores remotos correspondentes.
- Impacto: todos os clientes recebem uma visão consistente do mundo em tempo real.

Fluxo contínuo de dados

- O design já implementa um ciclo contínuo:
  - Cliente → envia posição a cada `_process()`.
  - Servidor → sempre responde com snapshot atualizado.
- Futuramente, isso pode ser otimizado via delta updates (enviar só mudanças) ou interpolação para suavizar movimento.

Mecanismo de nomenclatura do processo

Identificação dos componentes

- Cada jogador é identificado por um ID único no formato:

`"ip:porta"`
Esse ID é gerado pelo servidor a partir dos metadados do socket.

Registro e descoberta

- Cliente se registra explicitamente enviando `REGISTER:<nome>`.
- O servidor adiciona essa entrada em `players[id]`.
- O servidor inclui esse `id` em todas as mensagens `PLAYER_POSITIONS`, permitindo que outros clientes descubram jogadores automaticamente.

Informações armazenadas

Cada entrada no dicionário `players` contém:

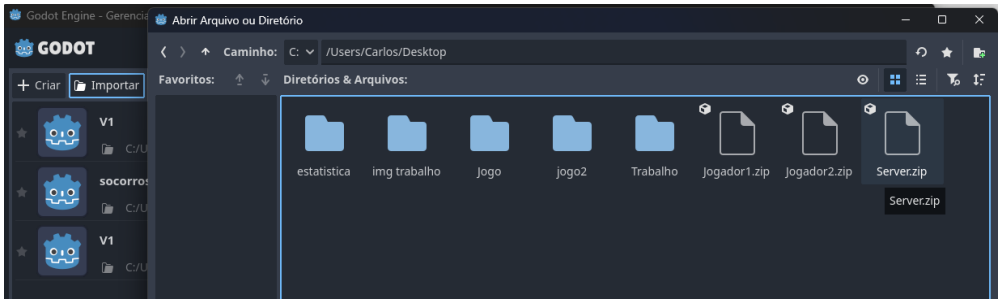
```
{
  "name": <string>,     # Nome definido no registro
  "position": Vector2,  # Última posição conhecida
  "ip": <string>,       # IP do cliente
  "port": <int>         # Porta do cliente
}
```

O cliente mantém um dicionário local `remote_players` que mapeia:

`"id" -> Node instanciado na cena`

Instruções de construção e execução

Abra uma aplicação para cada importe: Jogador 1, Jogador 2, Server



No canto esquerdo superior execute as aplicações. A aplicação Server deve ser executada primeiro

